

Module 15 - chapter 17

Project: Run Docker applications - Part 2

- The final YAML file used in the project can be found here: <https://gitlab.com/twn-devops-bootcamp/latest/15-ansible/ansible-projects/-/blob/main/deploy-docker-ec2-user.yaml>
- A copy of the MySQL can be found here: <https://gitlab.com/twn-devops-bootcamp/latest/15-ansible/bootcamp-java-mysql-project>

Ansible and Docker

Use "community.docker" ansible Collection

In the previous chapter we used 'command' module to run docker as a quick tests

```
44 - name: Test docker pull
45   hosts: docker_server
46   tasks:
47     - name: Pull redis
48       command: docker pull redis
49
```

which is the last resource when there are no other modules we can use to execute commands



command and *shell* - use only,
when there is no appropriate
Ansible module available

Because they don't have state
management

For docker there is a set of modules -> Docker Community collection

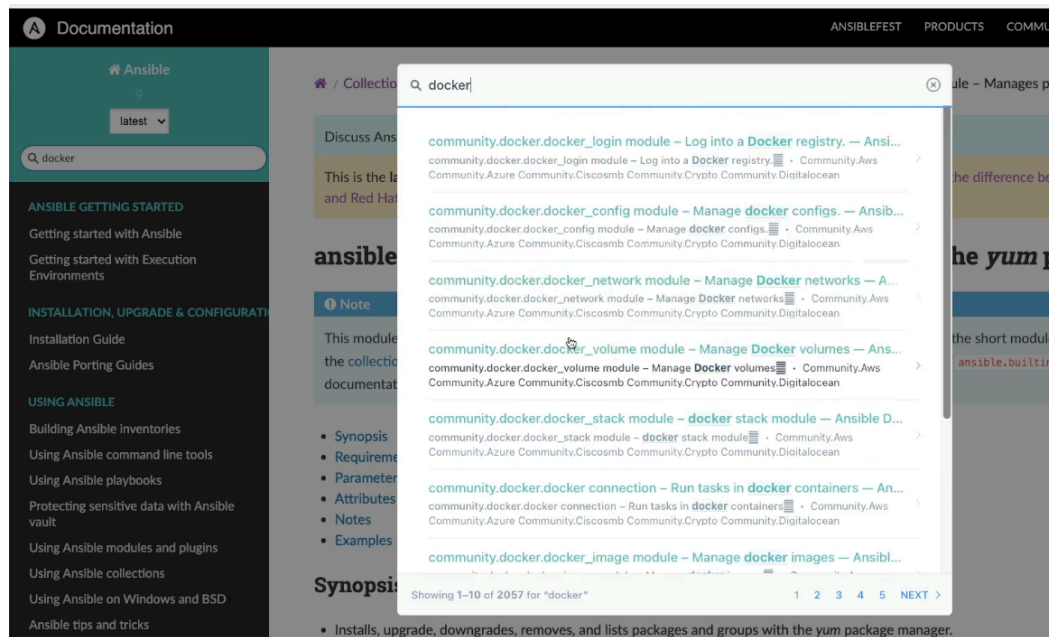
Docker Community Collection

- ▶ Includes many modules and plugins to
work with Docker



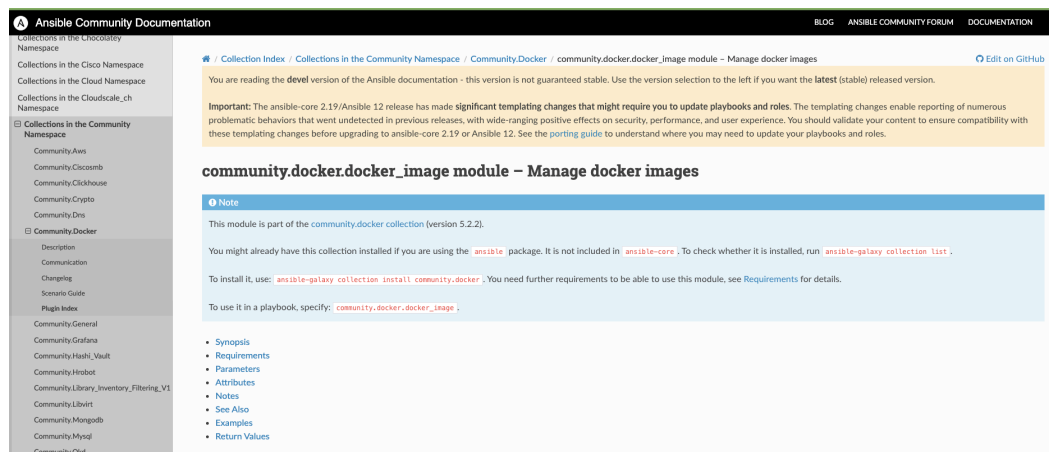
So in the documentation we can see modules related to docker

<https://docs.ansible.com/projects/ansible/devel/collections/community/docker/index.html#description>



Pull docker Image

https://docs.ansible.com/projects/ansible/devel/collections/community/docker/docker_image_module.html#ansible-collections-community-docker-docker-image-module



Docker images module is for pulling the images, building the images, pushing the images to a repo, etc.

We will re-write the 'command: docker pull redis' line and use the docker_image to pull the redis image

```

46 - name: test docker pull
47   hosts: docker_server
48   tasks:
49     - name: Pull redis
50       docker_image:
51         name: redis
52       source: pull

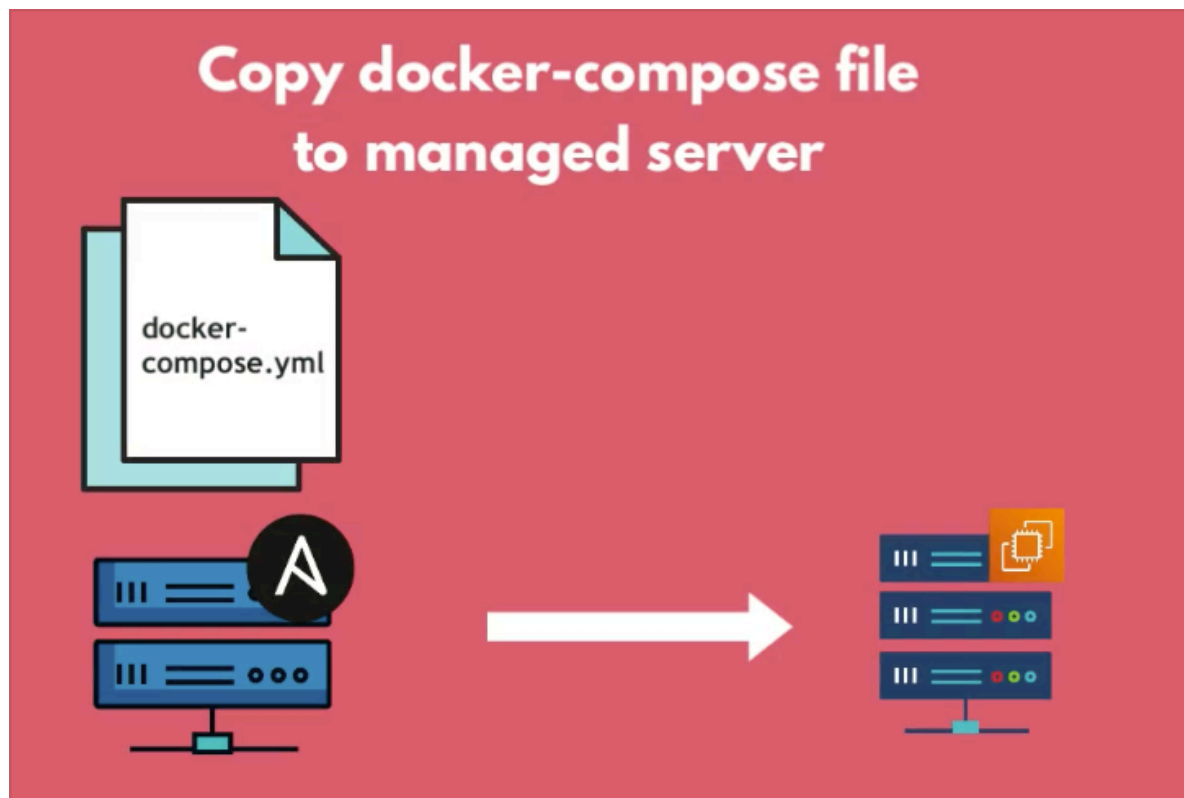
```

Write 5th play

Start docker containers

Now that we have tested the above, we don't need it any more, so we delete it and now we write the code that we actually need.

And the first task is to copy the docker-compose file from our local machine to the server



The docker-compose file contains the image names of the containers we want

to start on the server.

The docker-compose file is in a private repo

```
hosts M ! deploy-docker.yaml U docker-compose-full.yaml X
1 version: '3'
2 services:
3   java-app:
4     image: nanatwn/demo-app:java-maven-2.0
5     environment:
6       - DB_USER=user
7       - DB_PWD=pass
8       - DB_SERVER=mysql
9       - DB_NAME=my-app-db
10    ports:
11      - 8080:8080
```

Private Docker repository

Nana's docker hub private repo:

General Tags Builds Collaborators Webhooks Settings

nanatwn/demo-app 
Updated 22 days ago
This repository does not have a description 

Docker commands
To push a new tag to this repository:
`docker push nanatwn/demo-app:tagname`

Tags
This repository contains 25+ tag(s).

Tag	OS	Type	Pulled	Pushed
mysql-app		Image	6 days ago	22 days ago
jma-2.0		Image	---	3 months ago
java-maven-2.0		Image	3 hours ago	3 months ago
1.1.10-1		Image	---	4 months ago

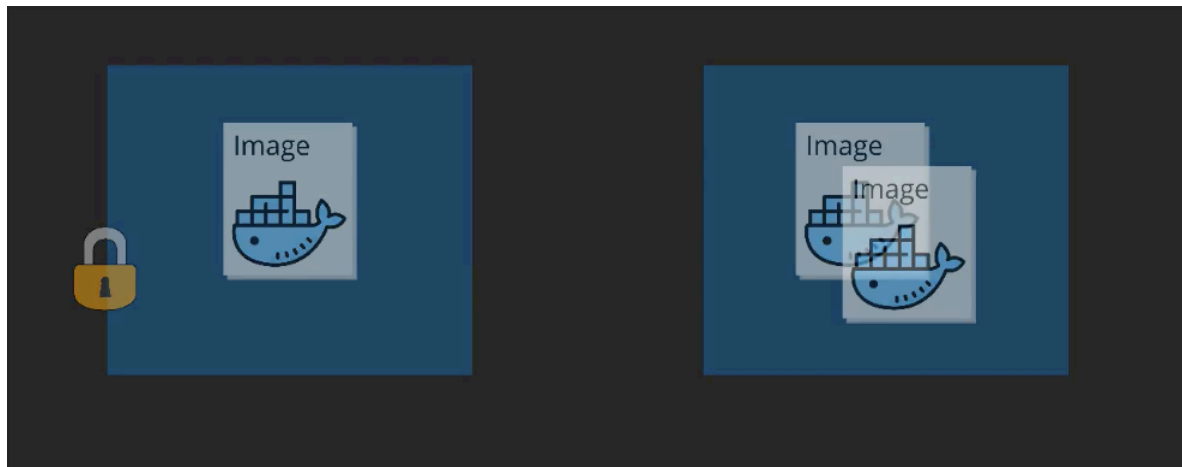
Automated Builds
Manually pushing images to Hub? Connect your account to Bitbucket to automatically build and tag new images when code is updated, so you can focus your time on creating.
Available with Pro, Team and Business subscriptions. [Read automated builds](#) 
[Upgrade](#)

And also is in the local machine 'docker-compose-full.yaml'

```
nana@macbook /Users/nana/bootcamp-java-mysql-project
% ls
Dockerfile                docker-compose-full.yaml  gradlew.bat
README.md                 docker-compose.yaml       settings.gradle
build.gradle              gradle                    src
deployment.yaml          gradlew
nana@macbook /Users/nana/bootcamp-java-mysql-project
%
```

'docker-compose-full' contains 3 images:

- One image is private -> the java app
- Two images are public -> mysql and phpadmin



This is Nana's code with the full path to her docker-compose file in line 49:

```
44 - name: Start docker containers
45   hosts: docker_server
46   tasks:
47     - name: Copy docker compose
48       copy:
49         src: /Users/nana/bootcamp-java-mysql-project/docker-compose-full.yaml
50         dest: /home/ec2-user/
51
```

Now is my turn:

- Cloned Nana's repo <https://gitlab.com/twn-devops-bootcamp/latest/15-ansible/bootcamp-java-mysql-project>
- Then I replaced grade for maven
- Then I broke the link to Nana's repo and create a new repo from the start:

Option B — full fresh start, no history at all (destructive):

```
-> cd /path/to/project
-> rm -rf .git
-> git init
-> git add .
-> git commit -m "Initial commit"
```

This wipes all commit history and starts a brand-new repo. Only do this if you don't need the original history.

- Then I created a new repo in my GitHub account using gh
- ```
-> gh repo create <repo-name> --private --source=. --remote=origin
```

- Then I build the app
- ```
-> mvn clean package
```

- Then create the java image for this demo

-> `docker build -t demo-app:java-maven-ansible-2.0 .`

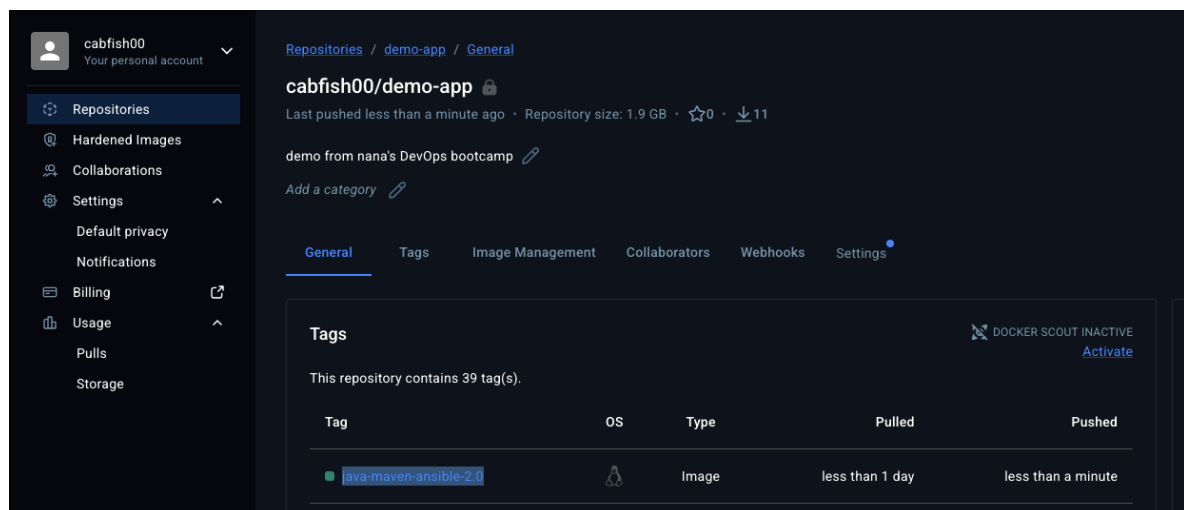
- Tag the image

-> `docker tag demo-app:java-maven-ansible-2.0 cabfish00/demo-app:java-maven-ansible-2.0`

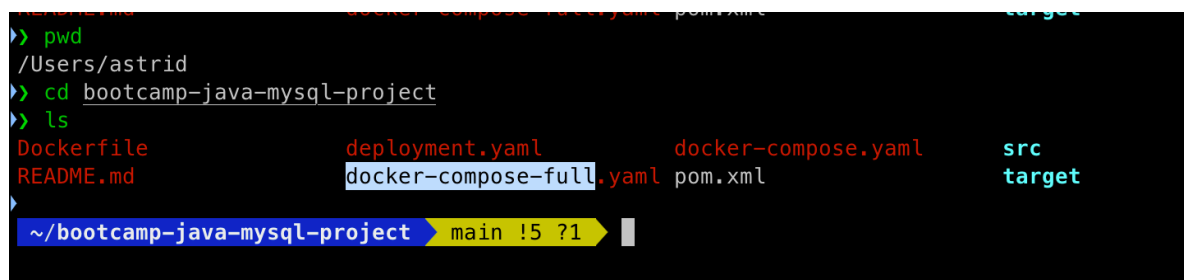
- Then push the tagged image to docker hub

-> `docker push cabfish00/demo-app:java-maven-ansible-2.0`

And it is in the docker hub repo:



- I check for the full local path where the docker-compose-full.yaml file is



- Copy the full local path as the source -> src to be used in the play:

-> `/Users/astrid/bootcamp-java-mysql-project/docker-compose-full.yaml`

```

46 - name: Start docker containers
47   hosts: docker_server
48   tasks:
49     - name: Copy docker compose
50       copy:
51         src: /Users/astrid/bootcamp-java-mysql-project/docker-compose-full.yaml
52         dest:
53

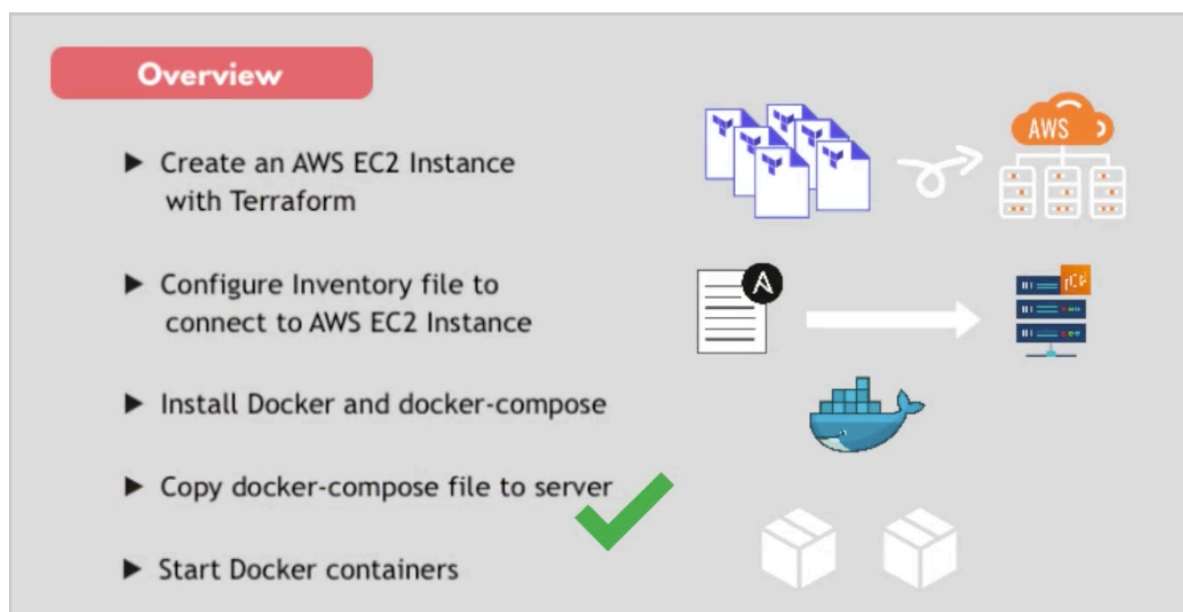
```

- And then we tell ansible where to copy the .yaml file in the remote server along with the name we want the file to have in the server

```

46 - name: Start docker containers
47   hosts: docker_server
48   tasks:
49     - name: Copy docker compose
50       copy:
51         src: /Users/astrid/bootcamp-java-mysql-project/docker-compose-full.yaml
52         dest: /home/ec2-user/docker-compose.yaml

```



Our server already has docker-compose available (we previously execute ansible playbook and it installed docker-compose).

But we can't use it just yet because our docker-compose file needs to pull an image from my private docker hub repo and for ansible to be able to do that it needs to login first or authenticate first with docker hub.

Try to pull the image into the EC2 instance directly from the server and it failed

as expected:

```
Run 'docker compose COMMAND --help' for more information on a command.
[ec2-user@ip-10-0-10-191 ~]$ docker pull cabfish00/demo-app:java-maven-ansible-2.0
Error response from daemon: pull access denied for cabfish00/demo-app, repository does not exist or may require 'docker login': denied: request
ed access to the resource is denied
[ec2-user@ip-10-0-10-191 ~]$
```

So we need ansible to execute docker login on the EC2. And for that we will use a docker module called 'docker_login'

https://docs.ansible.com/projects/ansible/devel/collections/community/docker/docker_login_module.html#ansible-collections-community-docker-docker-login-module

docker_login Module

- ▶ Similar to "docker login" command
- ▶ Authenticates with a docker registry and adds the credentials to your local Docker config file

And we can provide the 'registry_url' parameter

registry_url aliases: registry, url string	The registry URL. Default: <code>"https://index.docker.io/v1/"</code>
state 🔗 string	This controls the current state of the user. <code>present</code> will I To logout you only need the registry server, which default

but because our repo is docker which is the default value for this parameter then we don't need to use this parameter in our play

Wo we only need to provide the parameters for username and password:

```
56      - name: Docker login
57        docker_login:
58          username: cabfish00
59          password:
```

And best practice we don't hardcode a password instead we use a variable

```

46 - name: Start docker containers
47   hosts: docker_server
48   vars_files:
49     - project-vars
50   tasks:
51     - name: Copy docker compose
52       copy:
53         src: /Users/astrid/bootcamp-java-mysql-project/docker-compose-full.yaml
54         dest: /home/ec2-user/docker-compose.yaml
55     - name: Docker login
56       docker_login:
57         username: cabfish00
58         password: "{{docker_password}}"
59

```

Another way to set a variable's value is using a variable prompt -> so the playbook can prompt the user to enter the value

Interactive input: prompts

- ▶ Playbook prompts the user for certain input
- ▶ Prompting the user for variables lets you avoid recording sensitive data like passwords
- ▶ You could also encrypt the entered values

And also I can encrypt the entered values.

I refactor and now the code looks like this:

And I execute the playbook

```
46  - name: Start docker containers
47    hosts: docker_server
48    vars_prompt:
49      - name: docker_password
50        prompt: Enter password for docker registry
51    tasks:
52      - name: Copy docker compose
53        copy:
54          src: /Users/astrid/bootcamp-java-mysql-project/docker-compose-
55          dest: /home/ec2-user/docker-compose.yaml
56      - name: Docker login
57        docker_login:
58          username: cabfish00
59          password: "{{docker_password}}"
60
```

Document 1/1 > Item 4/4 > tasks: > Item 2/2 > docker_login: > username: > cabfish00

Terminal: Local x + v

```
TASK [Add ec2-user to docker group] *****
ok: [3.10.140.80]

TASK [Reconnect to server session] *****
Enter password for docker registry:
```

If I input the password

```
TASK [Reconnect to server session] *****
Enter password for docker registry:

PLAY [Start docker containers] *****

TASK [Gathering Facts] *****
ok: [3.10.140.80]

TASK [Copy docker compose] *****
ok: [3.10.140.80]

TASK [Docker login] *****
changed: [3.10.140.80]

PLAY RECAP *****
3.10.140.80 : ok=12 changed=2 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

And then I check the server, I can now download from my private registry

```

[ec2-user@ip-10-0-10-191 ~]$ docker pull cabfish00/demo-app:java-maven-ansible-2.0
java-maven-ansible-2.0: Pulling from cabfish00/demo-app
55afa1ecc21d: Pull complete
d27e94e76fff: Pull complete
f80aa99e53ae: Pull complete
f738373ee35e: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:705c40069317e555e3d15ba88a916696a2fc4c39a67a42a48b89850313b782d4
Status: Downloaded newer image for cabfish00/demo-app:java-maven-ansible-2.0
docker.io/cabfish00/demo-app:java-maven-ansible-2.0
[ec2-user@ip-10-0-10-191 ~]$

```

But for this demo we will store the password in a variables file:

```

46  - name: Start docker containers
47      hosts: docker_server
48      vars_files:
49          - project-vars
50      tasks:
51          - name: Copy docker compose
52              copy:
53                  src: /Users/astrid/bootcamp-java-mysql-project/docker-compose-full.yaml
54                  dest: /home/ec2-user/docker-compose.yaml
55          - name: Docker login
56              docker_login:
57                  username: cabfish00
58                  password: "{{docker_password}}"
59

```

And execute the playbook again
 -> ansible-playbook deploy-docker.yaml

```

TASK [Gathering Facts] *****
ok: [3.10.140.80]

TASK [Copy docker compose] *****
ok: [3.10.140.80]

TASK [Docker login] *****
ok: [3.10.140.80]

PLAY RECAP *****
3.10.140.80      : ok=12  changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

```

And it's working!

Our EC2 instance has now the docker-compose.yaml copied into it.

```
[ec2-user@ip-10-0-10-191 ~]$ ls  
docker-compose.yml
```

Next step is to execute the docker-compose file. We will translate the command below to ansible

```
[ec2-user@ip-10-0-10-186 ~]$ docker compose -f docker-compose.yml up
```

We will use the module 'docker_compose_v2'

The screenshot shows the Ansible Community Documentation website. The left sidebar lists various community collections, with 'Community.Docker' expanded. The main content area is titled 'These are the plugins in the community.docker collection:' and lists several modules. The 'docker_compose_v2 module' is highlighted, with a description: 'Manage multi-container Docker applications with Docker Compose CLI plugin'. Other modules listed include 'current_container_facts', 'docker_compose_v2_exec', 'docker_compose_v2_pull', 'docker_compose_v2_run', 'docker_config', 'docker_container', 'docker_container_copy_into', 'docker_container_exec', 'docker_container_info', and 'docker_context_info'.

This module has two versions so we need to use the full module name in this case

Examples

```
---
# Examples use the django example at https://docs.docker.com/compose/django. Follow it to create the
# flask directory

- name: Run using a project directory
  hosts: localhost
  gather_facts: false
  tasks:
    - name: Tear down existing services
      community.docker.docker_compose_v2:
        project_src: flask
        state: absent

    - name: Create and start services
      community.docker.docker_compose_v2:
        project_src: flask
        register: output

    - name: Show results
      ansible.builtin.debug:
        var: output

    - name: Run `docker compose up` again
      community.docker.docker_compose_v2:
        project_src: flask
        register: output

    - name: Show results
      ansible.builtin.debug:
        var: output

- ansible.builtin.assert:
```

And the code so far is:

```
59 - name: Start containers from compose
60   community.docker.docker_compose_v2:
61     project_src: /home/ec2-user
62     state: present # equivalent to 'docker compose up'
63
```

If the state is 'absent' that is the equivalent for 'docker compose down'

And 'present' is the default value for 'state' so we don't need to write it in the play as it is redundant

```
59 - name: Start containers from compose
60   community.docker.docker_compose_v2:
61     project_src: /home/ec2-user
```

I make sure I don't have images in the server (I deleted the image I pulled manually)

```
[ec2-user@ip-10-0-10-191 ~]$ docker images
REPOSITORY      TAG          IMAGE ID       CREATED        SIZE
[ec2-user@ip-10-0-10-191 ~]$
```

Now I execute the playbook
-> ansible-playbook deploy-docker.yaml

```
TASK [Docker login] *****
ok: [3.10.140.80]

TASK [Start containers from compose] *****
[WARNING]: Docker compose: unknown None: /home/ec2-user/docker-compose.yaml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
changed: [3.10.140.80]

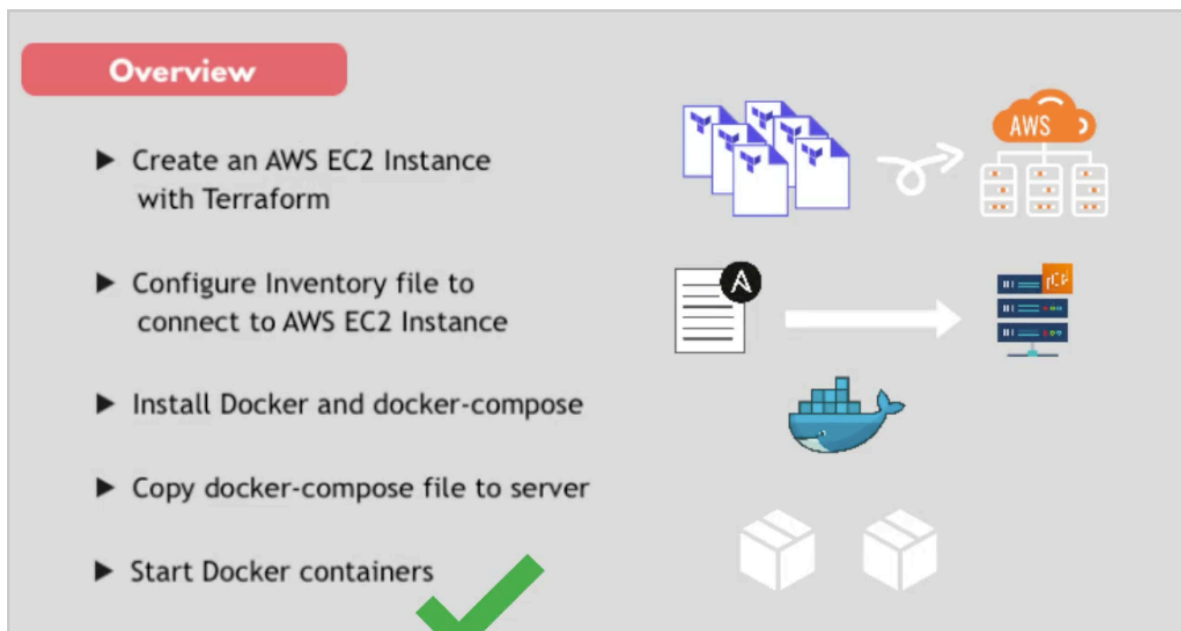
PLAY RECAP *****
3.10.140.80      : ok=13   changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

I check the EC2 instance
-> docker images

```
[ec2-user@ip-10-0-10-191 ~]$ docker images
REPOSITORY      TAG          IMAGE ID       CREATED        SIZE
cabfish00/demo-app  java-maven-ansible-2.0  7cf34ea3f318  2 hours ago   320MB
phpmyadmin        latest       00513f2d1d1b  4 days ago    575MB
mysql             latest       d236310860c6  12 days ago   945MB
```

-> docker ps

```
[ec2-user@ip-10-0-10-191 ~]$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
ed1dd063dd5c  phpmyadmin "/docker-entrypoint..." 19 seconds ago Up 17 seconds 0.0.0.0:8083->80/tcp, :::8083->80/tcp myadmin
1c3b7f729226  mysql     "docker-entrypoint.s..." 19 seconds ago Up 17 seconds 0.0.0.0:3306->3306/tcp, :::3306->3306/tcp, 33060/tcp mysql
```



Execute playbook on a new server

So I will destroy the current terraform set up

-> terraform destroy

```
aws_vpc.myapp-vpc: Destroying... [id=vpc-041bece5b00f793e]
aws_vpc.myapp-vpc: Destruction complete after 0s

Destroy complete! Resources: 7 destroyed.

~/Pyt/Ansible ▶ Module_15/An...16_Terraform *1 ?3 ▶
```

Instances (1/1) [Info](#)

Saved filter sets: Choose filter set ▼

Find Instance by attribute or tag (case-sensitive)

☒	Name ↗	Instance ID	Instance state ▼	Instance type ▼	Status check ↕	Availability Zone ▼	Pub
☒	dev-server-1	i-071af71d1c0b07e49	Terminated 🔍	t3.small	...	eu-west-2a	-

Loading... (1 processed)

And I create a new one

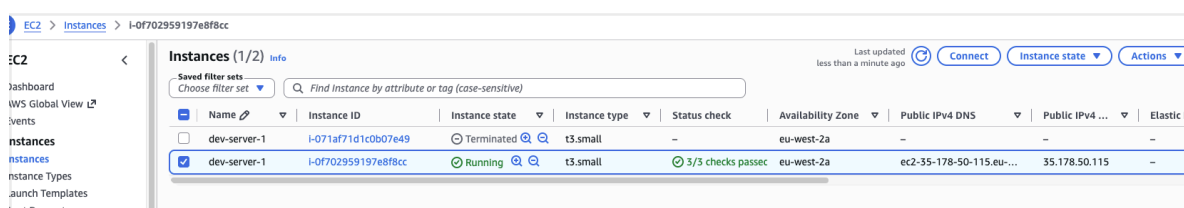
-> terraform apply


```
Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

aws-ami_id = "ami-01c952cfc86b7870d"
ec2-public_ip_1 = "35.178.50.115"

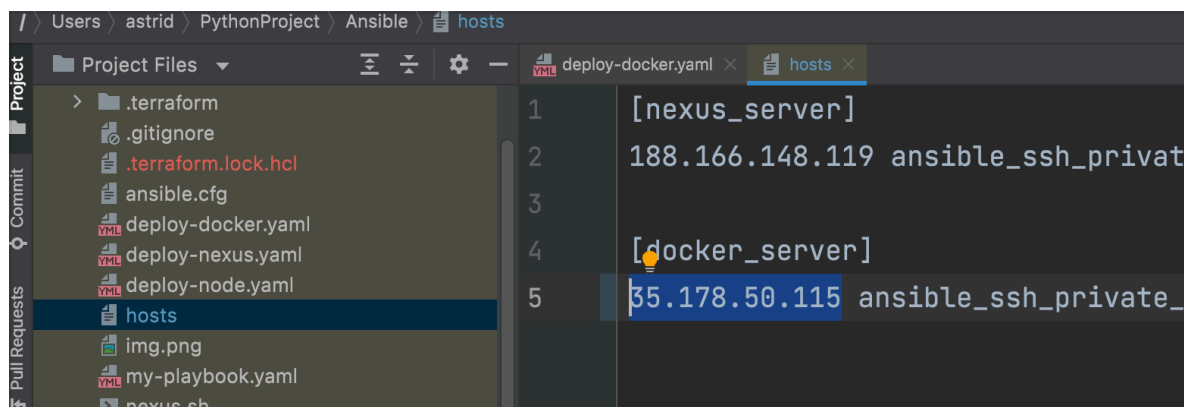
~/Pyt/Ansible ▶ Module_15/An...16_Terraform *1 ?3 ▶
```



The screenshot shows the AWS Management Console 'Instances' page. A table lists two EC2 instances. The first instance, 'dev-server-1', is in a 'Terminated' state. The second instance, also named 'dev-server-1', is in a 'Running' state. The running instance has a public IP address of 35.178.50.115.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP
dev-server-1	i-071af71d1c0b07e49	Terminated	t3.small	-	eu-west-2a	-	-	-
dev-server-1	i-0f702959197e8f8cc	Running	t3.small	3/3 checks passed	eu-west-2a	ec2-35-178-50-115.eu-...	35.178.50.115	-

And now I use the new Ip address of the new remote server and update the playbook



The screenshot shows an IDE with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with files like .gitignore, .terraform.lock.hcl, ansible.cfg, and several .yaml files. The code editor shows the 'hosts' file with two host entries: 'nexus_server' with IP 188.166.148.119 and 'docker_server' with IP 35.178.50.115.

```
1 [nexus_server]
2 188.166.148.119 ansible_ssh_privat
3
4 [docker_server]
5 35.178.50.115 ansible_ssh_private_
```

And now when we execute the ansible playbook, the Ec2 instance will be fully configured with 3 running containers:
-> ansible-playbook deploy-docker.yaml

```
TASK [Start containers from compose] *****
[WARNING]: Docker compose: unknown None: /home/ec2-user/docker-compose.yaml: the attribute `version` is obsolete, it will b
nored, please remove it to avoid potential confusion
changed: [35.178.50.115]

PLAY RECAP *****
35.178.50.115      : ok=13  changed=9  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

I check the server

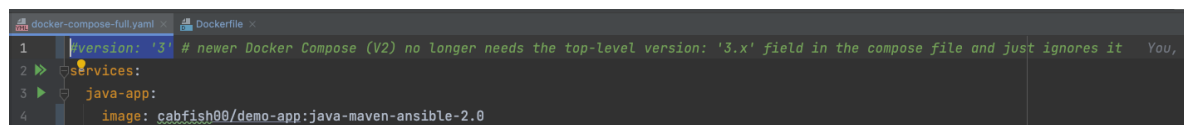
Connection bean is created eagerly), gets "Connection refused" because MySQL's port isn't listening yet, throws, and the whole app crashes and exits.

I need to fix the docker-compose file.

First I will get rid of the version so I don't get the ansible warning

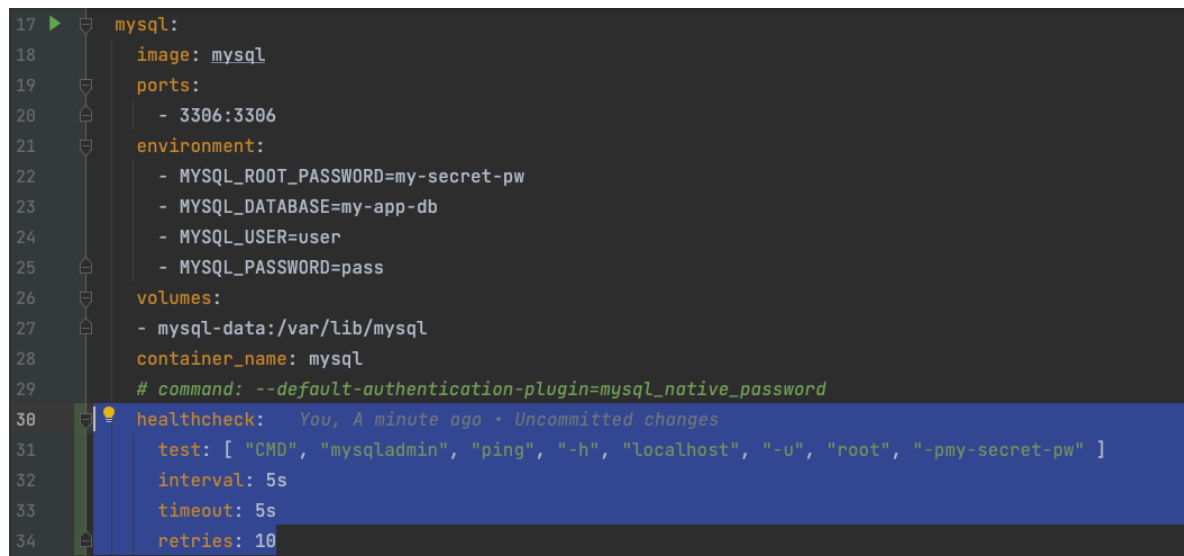
```
TASK [Start containers from compose] *****
[WARNING]: Docker compose: unknown None: /home/ec2-user/docker-compose.yaml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
changed: [35.178.50.115]

PLAY RECAP *****
35.178.50.115      : ok=13   changed=9    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```



```
1 |version: '3' # newer Docker Compose (V2) no longer needs the top-level version: '3.x' field in the compose file and just ignores it
2 |services:
3 |  java-app:
4 |    image: cabfish00/demo-app:java-maven-ansible-2.0
```

Next, add a healthcheck to mysql



```
17 | mysql:
18 |   image: mysql
19 |   ports:
20 |     - 3306:3306
21 |   environment:
22 |     - MYSQL_ROOT_PASSWORD=my-secret-pw
23 |     - MYSQL_DATABASE=my-app-db
24 |     - MYSQL_USER=user
25 |     - MYSQL_PASSWORD=pass
26 |   volumes:
27 |     - mysql-data:/var/lib/mysql
28 |   container_name: mysql
29 |   # command: --default-authentication-plugin=mysql_native_password
30 |   healthcheck:
31 |     test: [ "CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-pmy-secret-pw" ]
32 |     interval: 5s
33 |     timeout: 5s
34 |     retries: 10
```

Next, make java-app wait for mysql to be healthy, plus a restart policy as a safety net:

```

2  >> services:
3  >  java-app:
4      image: cabfish00/demo-app:java-maven-ansible-2.0
5      environment:
6          - DB_USER=user
7          - DB_PWD=pass
8          - DB_SERVER=mysql
9          - DB_NAME=my-app-db
10     ports:
11         - 8080:8080
12     container_name: my-java-app
13     depends_on: You, Moments ago • Uncommitted changes
14         mysql:
15             condition: service_healthy
16             restart: on-failure
17 >  mysql:

```

I created a docker image from the .jar file so the image doesn't contain the docker-compose file(s). So I don't need to create the docker image again.

All I need is to execute the playbook again. The docker-compose-full.yaml file has been updated and ansible will realise this with the module 'copy' because this module is idempotent.

Ansible copies the new version over and overwrites the file on the server, showing changed in the task output. If the file were identical, it would just report ok and skip the actual copy.

-> ansible-playbook deploy-docker.yaml

```

TASK [Copy docker compose] *****
changed: [35.178.50.115]

TASK [Docker login] *****
ok: [35.178.50.115]

TASK [Start containers from compose] *****
changed: [35.178.50.115]

PLAY RECAP *****
35.178.50.115      : ok=13  changed=3  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

```

So the 'copy' module detected the change

And now I check the EC2 instance

```
[ec2-user@ip-10-0-10-160 ~]$ docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
30558f7f44c5	cabfish00/demo-app:java-maven-ansible-2.0		"java -jar bootcamp-..."	2 minutes ago	Up 2 minutes	0.0.0.0:8080->8080/tcp, :::8080->8080/tcp
d67e594621e5	mysql	my-java-app	"docker-entrypoint.s..."	2 minutes ago	Up 2 minutes (healthy)	0.0.0.0:3306->3306/tcp, :::3306->3306/tcp
8325ada3a6e0	phpmyadmin	myadmin	"/docker-entrypoint..."	29 minutes ago	Up 29 minutes	0.0.0.0:8083->8083/tcp, :::8083->8083/tcp

And the java app is running now 🥳

Make the Playbook more generic

If we want to execute this ansible playbook in another server that has a different linux distribution.

For the EC2 instance we are using 'yum' package manager and the user 'ec2-user'



```
! deploy-docker.yaml U X
name: Install Docker-compose
tasks:

name: Add ec2-user to docker group
hosts: docker_server
become: yes
tasks:
- name: Add ec2-user to docker group
  user:
    name:
    group:
    append: yes
```

Try to make the playbook as re-usable as possible!

So instead of using 'ec2-user' we will create a new user and this new user will execute ansible, making the playbook more general purpose.

So I created a new file with the same content as 'deploy-docker.yaml' and called it 'deploy-docker-new-user.yaml' and we will remove lines 38 - 44

```
28 - name: Install docker-compose
29   get_url:
30     url: https://github.com/docker/compose/releases/download
31     dest: ~/.docker/cli-plugins/docker-compose
32     mode: +x
33
34 - name: Add ec2-user to docker group
35   hosts: docker_server
36   become: yes
37   tasks:
38     - name: Add ec2-user to docker group
39       user:
40         name: ec2-user
41         groups: docker
42         append: yes
43     - name: Reconnect to server session
44       meta: reset_connection
45
46 - name: Start docker containers
47   hosts: docker_server
```

And we will rename the play in line 34

```
34 - name: Create new linux user
35   hosts: docker_server
36   become: yes
37   tasks:
38     - name: Create new linux user
39       user:
40         name: astrid
41         group: adm,docker
```

And we will add the user 'astrid' to the groups: adm (admin in AWS) and docker.

And we don't have to reset the connection like we did before because we are creating a new user (not adding an existing one to a group) and we are adding the user to a group(s) on creation.

And move the play to create a new user after installing docker and before the other plays

```
1  ---
2  - name: Install Docker
3    hosts: docker_server
4    become: yes
5    tasks:
6      - name: Make sure Docker is installed
7        yum:
8          name: docker
9          update_cache: yes
10         state: present
11      - name: Ensure Docker is running
12        systemd:
13          name: docker
14          state: started
15
16  - name: Create new linux user
17    hosts: docker_server
18    become: yes
19    tasks:
20      - name: Create new linux user
21        user:
22          name: astrid
23          group: adm,docker
```

Now we edit the other plays using the new user 'astrid'

```

24
25 - name: Install Docker-compose
26   hosts: docker_server
27   become: yes
28   become_user: astrid
29   vars_files:
30     - project-vars
31   tasks:
32     - name: Create docker-compose directory
33       file:
34         path: ~/.docker/cli-plugins
35         state: directory
36     - name: get architecture of remote machine
37       shell: uname -m
38       register: remote_arch
39     - name: Install docker-compose
40       get_url:
41         url: https://github.com/docker/compose/releases/download/v{{docker_compose_
42         dest: ~/.docker/cli-plugins/docker-compose
43         mode: +x

```

And here we add the user

```

45 - name: Start docker containers
46   hosts: docker_server
47   become: yes
48   become_user: astrid
49   vars_files:
50     - project-vars
51   tasks:
52     - name: Copy docker compose
53       copy:
54         src: /Users/astrid/bootcamp-java-mysql-project/docker-compose-full.yaml
55         dest: /home/ec2-user/docker-compose.yaml
56     - name: Docker login
57       docker_login:
58         username: cabfish00
59         password: "{{docker_password}}"
60     - name: Start containers from compose
61       community.docker.docker_compose_v2:
62         project_src: /home/ec2-user
63

```

And edit the path in the server from /home/ec2-user to /home/astrid


```

45 - name: Start docker containers
46   hosts: docker_server
47   become: yes
48   become_user: astrid
49   vars_files:
50     - project-vars
51   tasks:
52     - name: Copy docker compose
53       copy:
54         src: /Users/astrid/bootcamp-java-mysql-project/docker-compose-full.yaml
55         dest: /home/astrid/docker-compose.yaml
56     - name: Docker login
57       docker_login:
58         username: cabfish00
59         password: "{{docker_password}}"
60     - name: Start containers from compose
61       community.docker.docker_compose_v2:
62         project_src: /home/astrid
63

```

Now is more generic, but 'add' is still a problem as this is particular to AWS but other OS might use 'admin' instead of 'adm'

```

16 - name: Create new linux user
17   hosts: docker_server
18   become: yes
19   tasks:
20     - name: Create new linux user
21       user:
22         name: astrid
23         group: adm,docker
24

```

So we can use parameters to make it more generic (remember to add the vars_file reference)

```
16  - name: Create new linux user
17    hosts: docker_server
18    vars_files:
19      - project-vars
20    become: yes
21    tasks:
22      - name: Create new linux user
23        user:
24          name: astrid
25          group: "{{user_groups}}"
26
```